

(Group B)

Nafisa Ayman

Answer to the Q.No-03(a)

Hash Function: A hash function is any function that can be used to map data of arbitrary size to fixed-size values. The values returned by a hash function are called hash values, hash codes, digests or simply hashes. The values are used to index a fixed size table called a hash table.

Answer to the Q.No-03(b)

i) The complexity of linear search algorithm:

~~O(n)~~ It takes up no extra space; its space complexity is $O(1)$ for an array of n elements.

ii) The complexity of bubble sort algorithm:

It is a reliable sorting algorithm. This algorithm has a worst-case time complexity of $O(n^2)$. The bubble sort has a space complexity of $O(1)$.

iii) The complexity of Binary Search Algorithm: $O(\log n)$.

The best-case time complexity would be $O(1)$ when the central index would directly match the desired value.

iv) The complexity of Insertion Sort Algorithm: $O(n^2)$. The best-case time complexity of insertion sort algorithm is $O(n)$.

v) The complexity of Selection Sort Algorithm: $O(n^2)$. Selection Sort Algorithm has a space complexity of $O(1)$ because it necessitates some extra memory space for temp variable for swapping.

Answer to the Q.No-03(c)

We will apply selection sort in the given sorted array A and show each pass separately. Let's determine int k & pass the characters Array is A[10]

⊛ (arrow mark $\rightarrow$ swap)

Pass	Loc	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]
K=1	2	B	A	N	G	L	A	D	E	S	H
K=2	6	A	B	N	G	L	A	D	E	S	H
K=3	6	A	A	N	G	L	B	D	E	S	H
K=4	7	A	A	B	G	L	N	D	E	S	H
K=5	8	A	A	B	D	L	N	G	E	S	H
K=6	7	A	A	B	D	E	N	G	L	S	H
K=7	10	A	A	B	D	E	G	N	L	S	H
K=8	8	A	A	B	D	E	G	H	L	S	N
K=9	10	A	A	B	D	E	G	H	L	S	N
Sorted:		A	A	B	D	E	G	H	L	N	S

[Or]

We will apply selection sort in the given sorted array A & show each pass separately. Let's

determine first k & pass the characters Array is A[14].

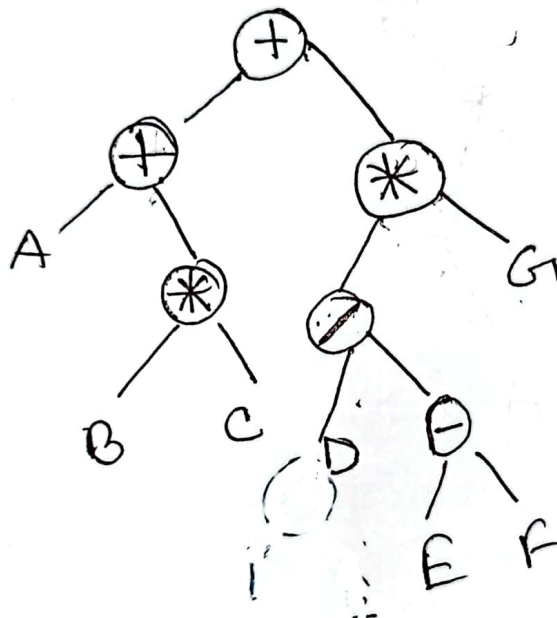
Pass	Loc	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]	A[10]	A[11]	A[12]	A[13]	A[14]
k=1	2	A	A	T	A	S	T	R	U	C	T	U	R	E	S
k=2	4	A	A	T	A	S	T	R	U	C	T	U	R	E	S
k=3	5	A	A	T	A	S	T	R	U	C	T	U	R	E	S
k=4	7	A	A	C	D	S	T	R	U	T	T	U	R	E	S
k=5	13	A	A	C	D	S	T	R	U	T	T	U	R	E	S
k=6	7	A	A	C	D	E	T	R	U	T	T	U	R	S	S
k=7	12	A	A	C	D	E	R	T	U	T	T	U	R	S	S
k=8	13	A	A	C	D	E	R	R	U	T	T	U	R	S	S
k=9	14	A	A	C	D	E	R	R	R	S	T	U	T	U	S
k=10	10	A	A	C	D	E	R	R	R	S	T	U	T	U	T
k=11	14	A	A	C	D	E	R	R	R	S	T	U	T	U	T
k=12	12	A	A	C	D	E	R	R	R	S	T	T	U	U	U
k=13	13	A	A	C	D	E	R	R	R	S	T	T	T	U	U
k=14	14	A	A	C	D	E	R	R	R	S	T	T	T	U	U

Sorted: A P C D E R R S S T T T U U

Spring 22

4a $\rightarrow$ Aut 18 (5a)

$$(A + B * C) + ((D / E - F) * G)$$



Preorder (Root - L - R)

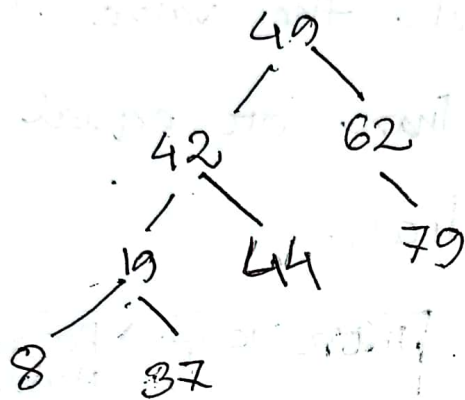
+ + A * B C * / D - E F G

Inorder (L - Root - R)

A + B * C + D / - E F * G

order: (L-R-Root)

4a) A B C * + D E F - / G * + /
① 49 42 19 62 37 79 44 8



Binary Search Tree:

It is a rooted binary tree with the key of each internal node being greater than all the keys in the respective

nodes left subtree & less than the ones in its right subtree

② Inorder traversal of the above tree:

L → Root → R.

8, 19, 37, 42, 44, 49, 62, 79.

4b) ① min heap. because min heap of size 10 is sufficient to find the top 10 largest items.

Or,

$\left\{ \begin{array}{l} \text{বড় হলে} \rightarrow \text{ফান} \\ \text{ছোট হলে} \rightarrow \text{বান্ন} \end{array} \right.$

Max-Heap: A max heap is a complete binary tree in which the value of a ~~tree~~ node is greater than or equal to the values of its children.

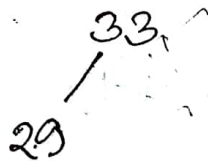
max heap	→	Parent Node	<	Child node
min heap	→	Parent Node	>	Child node

33, 29, 49, 21, 57, 62, 73, 54.

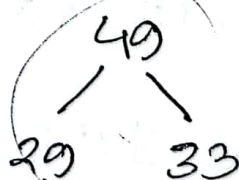
(i) item = 33

(33)

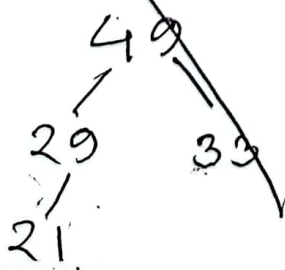
(ii) item = 29



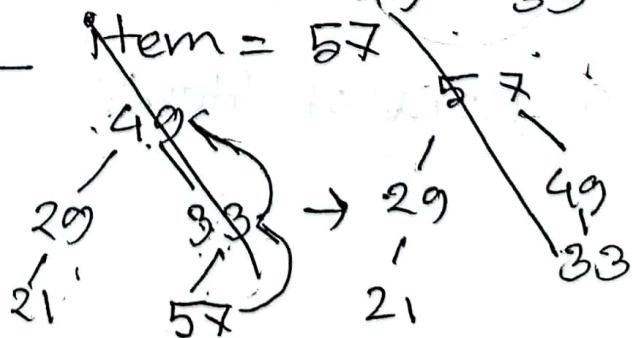
(iii) item = 49



(iv) item = 21

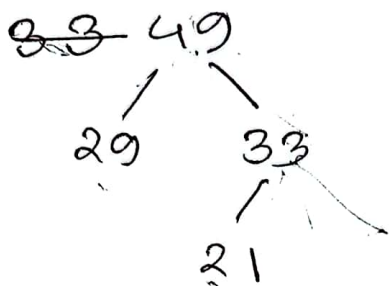


(v) item = 57

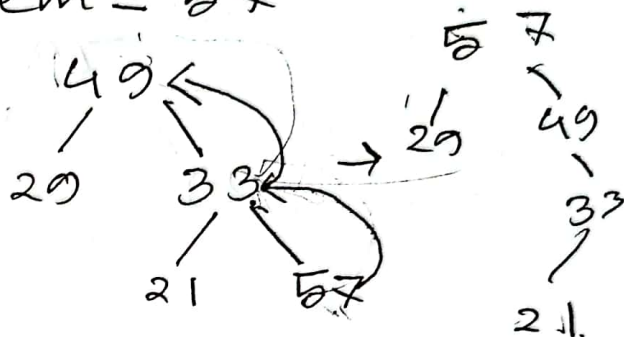


~~ND~~ item = 62.

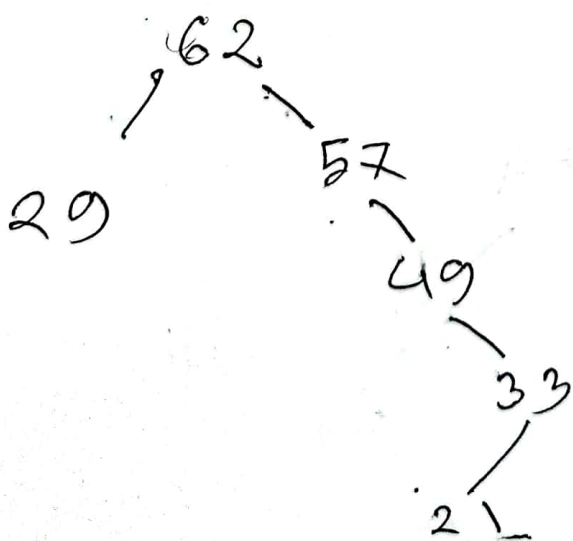
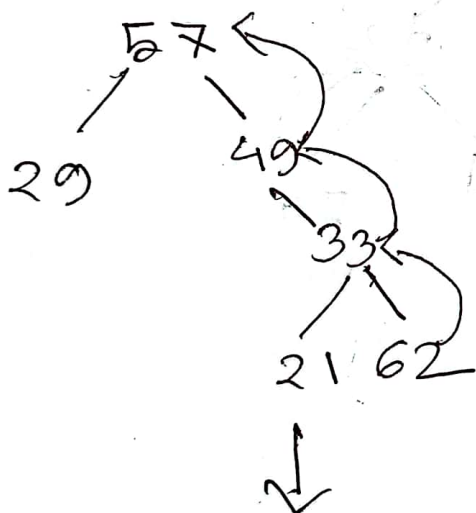
(N) item = 21



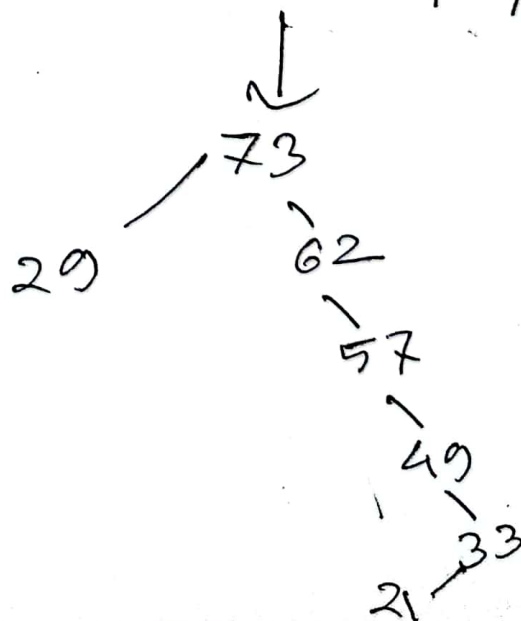
(N) item = 57



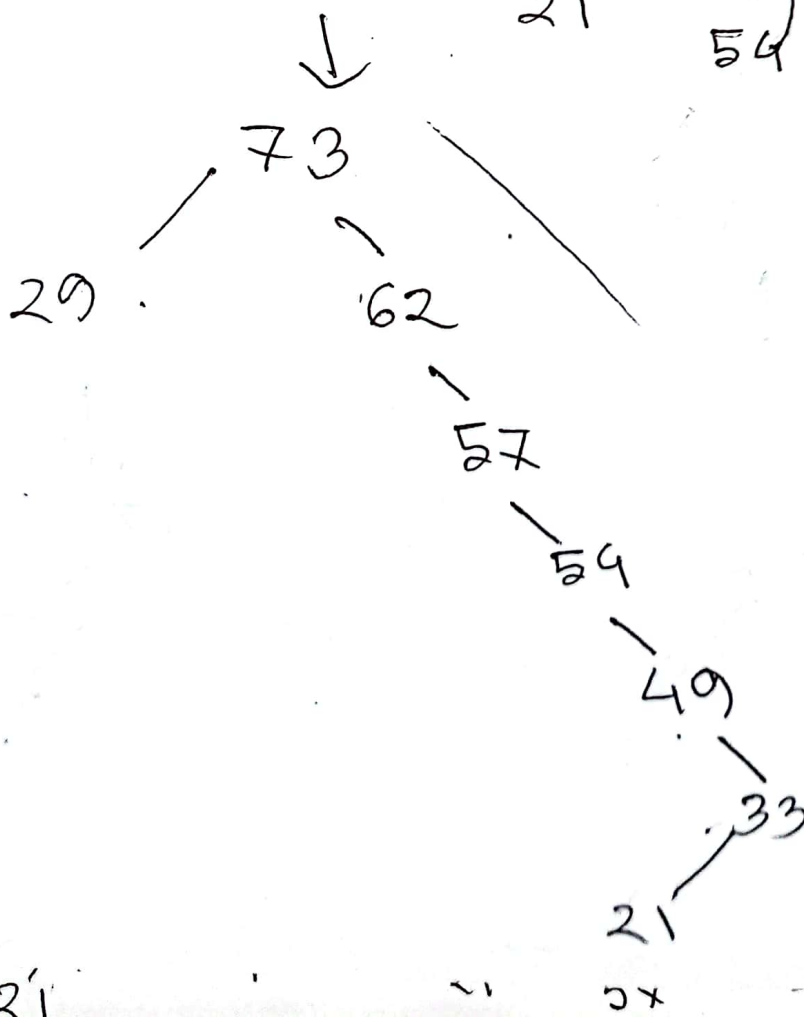
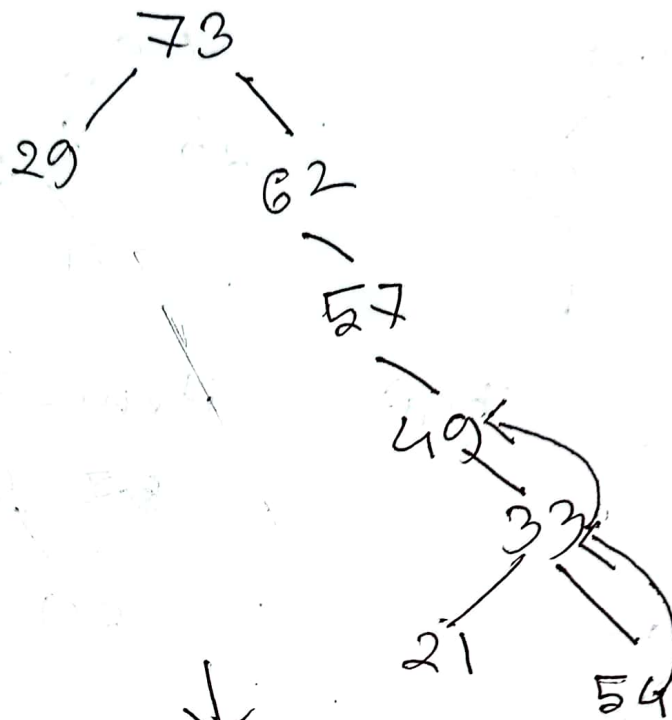
ND item = 62 (N11)



item = 73



(viii) item = 54



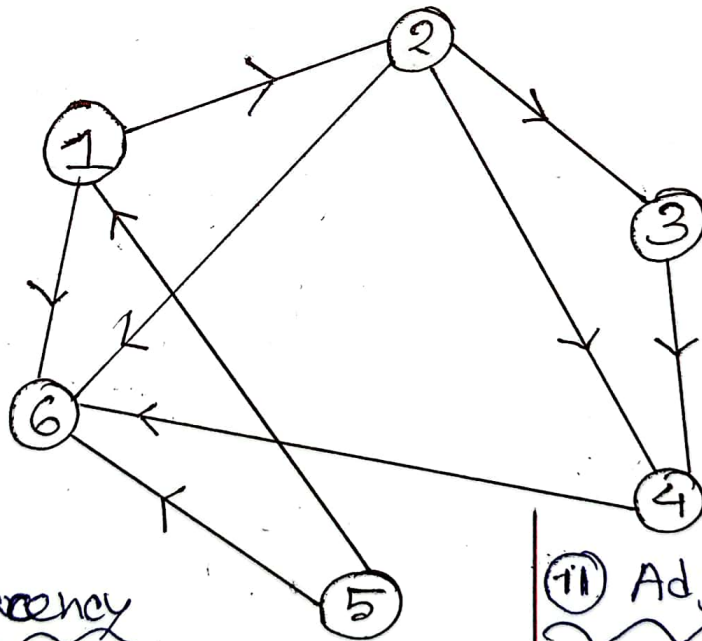
Answer to the Q.No-05(a)

Here, $G = (V, E)$

$$V(G) = \{1, 2, 3, 4, 5, 6\}$$

$$E(G) = \{(1, 2), (2, 3), (3, 4), (5, 1), (5, 6), (2, 6), (1, 6), (4, 6), (2, 4)\}$$

Now, using the ^{specified} above drawing the graph:



① Adjacency Matrix

	1	2	3	4	5	6
1	0	1	0	0	0	1
2	0	0	1	1	0	1
3	0	0	0	1	0	0
4	0	0	0	0	0	1
5	1	0	0	0	0	1
6	0	0	0	0	0	0

① Adjacency list

1		→	2		→	6	X			
2		→	3		→	4		→	6	X
3		→	4	X						
4		→	6	X						
5		→	1		→	6	X			
6	X									

Answer to the Q.No:- 05 (b)

For a set of vertices V with n elements $n(n-1)/2$ possible edges there. The graph containing maximum number of edge in a n element undirected graph without self loop is complete graph.

Answer to the Q.No-05(c)

A comparison between breadth first search (BFS) & depth first search (DFS) is given below:

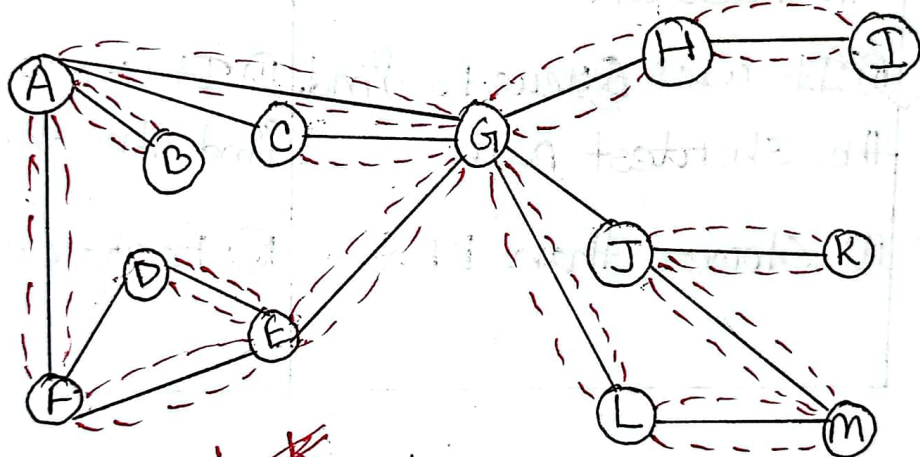
BFS	DFS
① BFS, stands for Breadth first search.	① DFS, stands for Depth first search
② It uses Queue to find the shortest path.	② It uses stack to find the shortest path
③ Slower than DFS	③ Faster than BFS

Ans. Q. 5(c)
Q. Number
(Q. 5(c))

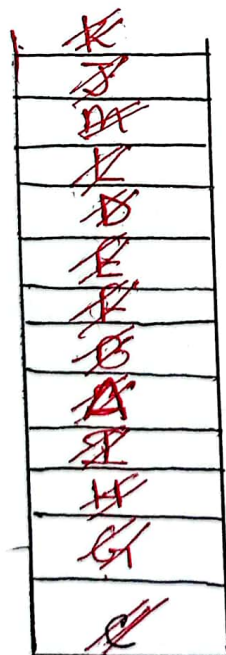
Answer to the Q.No - 05(d)

my ID is odd (C213202) so start from node C.

Constructing the depth first spanning Tree (DFS):



Stack:



Adjacet List:

A: G, C, B, F

B: A

C: A, G

D: F, E

E: F, G

F: A, D, E

G: A, C, H, E, L, J

H: G, I

I: H

J: G, K, M

K: J

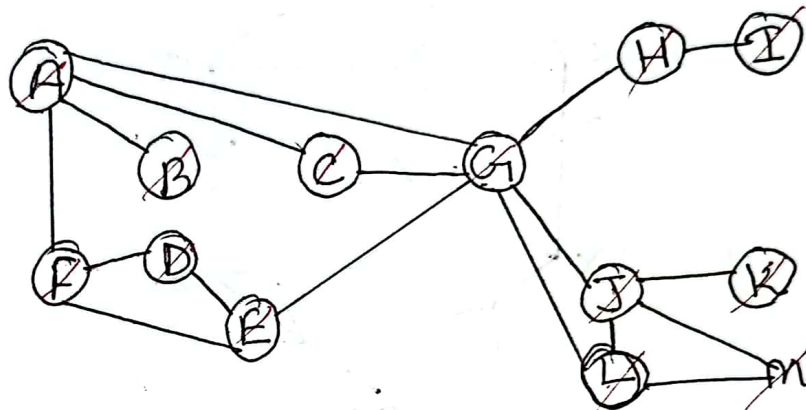
L: M, G

M: J, L

Result: C G H I A B F E D L M J K.

Constructing the breadth first spanning
Tree (BFS):

(FIFO)



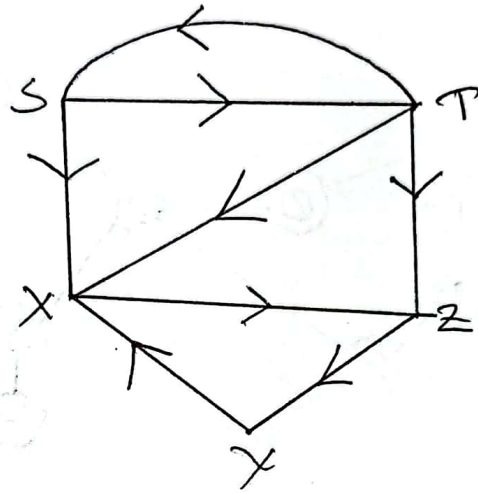
Queue:

C	A	H	B	F	E	A	J	D	I	K	M
---	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------

Result: C A G B F D E H I J K L M

Answer to the QNo-05 (Or)

①



	S	T	X	Y	Z
S	0	1	1	0	0
T	1	0	1	0	1
X	0	0	0	0	1
Y	0	0	1	0	0
Z	0	0	0	1	0

11

$$P_1(S, T) =$$

The Path matrix P of the Graph G:

	S	T	X	Y	Z
S	0	1	1	1	1
T	1	0	1	1	1
X	0	0	1	1	1
Y	0	0	1	1	1
Z	0	0	1	1	1